Tömb-metódusok – gyakorló feladatsor

JavaScript alapok · 10. fejezet · 26 feladat

A feladatsor a tömb-metódusok működését gyakoroltatja. Az elején minden metódushoz néhány nagyon egyszerű feladat tartozik — ezeknél az a cél, hogy megértsd, **mit kap a metódusnak átadott függvény, és mit ad vissza a metódus**. A végén néhány összetettebb feladat áll, amelyekben több metódust kell kombinálni. A feladatokat ciklus (`for`, `while`) nélkül oldd meg.

Minden feladat az alábbi három tömbbel dolgozik: másold be őket a fájlod elejére. Egy fájlba több feladat megoldását is írhatod egymás alá. Futtatás: `node feladatok.js`. Ahol a feladat nem ad külön mintát, ott az eredményt egyszerűen `console.log`-gal írd ki.

KÖZÖS KIINDULÓ ADATOK

```
const szamok = [4, 9, 16, 7, 12, 3];
const nevek = ['Anna', 'béla', 'Csilla', 'dávid'];
const termekek = [
  { id: 1, nev: 'Egér', ar: 6990, keszlet: 12, kategoria: 'periféria' },
  { id: 2, nev: 'Billentyűzet', ar: 14990, keszlet: 0, kategoria: 'periféria' },
  { id: 3, nev: 'Monitor', ar: 54990, keszlet: 3, kategoria: 'kijelző' },
  { id: 4, nev: 'HDMI-kábel', ar: 1990, keszlet: 40, kategoria: 'kábel' },
  { id: 5, nev: 'Webkamera', ar: 12990, keszlet: 0, kategoria: 'periféria' },
  { id: 6, nev: 'USB-C kábel', ar: 2490, keszlet: 25, kategoria: 'kábel' }
];
```

forEach — csinálj valamit minden elemmel

A `tomb.forEach(fn)` a tömb **minden elemére** meghívja a megadott függvényt, sorban. A függvény megkapja az aktuális elemet (és ha kéred, második paraméterként az indexét is). A `forEach` **nem ad vissza semmit** — akkor használjuk, ha csak „tenni” akarunk valamit az elemekkel, például kiírni őket.

1. Számok egyenként

Írd ki a `szamok` tömb minden elemét külön sorba.

2. Sorszámozott névsor

Írd ki a `nevek` tömb elemeit sorszámozva, soronként egyet. A sorszám 1-től induljon, a sor formája: `1. Anna`

map — alakíts át minden elemet

A `tomb.map(fn)` egy **új tömböt** ad vissza, amely **ugyanannyi elemű**, mint az eredeti: minden elem helyére az kerül, amit a függvény arra az elemre visszaad. Az eredeti tömb nem változik.

3. Duplázás

Készíts új tömböt, amelyben a `szamok` minden elemének a kétszerese szerepel. Írd ki az új tömböt.

4. Csupa nagybetű

Készíts új tömböt a `nevek` elemeiből úgy, hogy minden név csupa nagybetűs legyen. Írd ki az új tömböt.

5. Terméknevek

Készíts egy tömböt, amely csak a termékek **nevét** tartalmazza (objektumok helyett szövegeket). Írd ki.

6. Árcímkék

Készíts egy tömböt, amelyben minden termékhez egy szöveg tartozik ebben a formában: `Egér – 6990 Ft`. Írd ki a tömböt.

filter — válogasd ki, ami megfelel

A `tomb.filter(fn)` egy **új tömböt** ad vissza, amelybe csak azok az elemek kerülnek be, amelyekre a függvény **igazat** (`true`) ad vissza. Az elemek maguk nem változnak, csak kevesebb lesz belőlük (akár nulla is).

7. Tíznél nagyobbak

Válogasd ki a `szamok` tömbből a 10-nél nagyobb számokat egy új tömbbe, és írd ki.

8. Páros számok

Válogasd ki a `szamok` tömbből a páros számokat egy új tömbbe, és írd ki.

9. Kifogyott termékek

Válogasd ki azokat a termékeket, amelyekből nincs készleten (a készlet 0). Írd ki, hány ilyen termék van, majd magát a tömböt is.

find és findIndex — keresd meg az elsőt

A `tomb.find(fn)` az **első olyan elemet** adja vissza, amelyre a függvény igazat ad — magát az elemet, nem tömböt. Ha nincs ilyen, az eredmény `undefined`. A `tomb.findIndex(fn)` ugyanígy keres, de az elem **indexét** adja vissza, és ha nincs találat, `-1`-et.

10. Az első nagy szám

Keresd meg a `szamok` tömbben az első 10-nél nagyobb számot, és írd ki.

11. Termék azonosító alapján

Keresd meg azt a terméket, amelynek az `id`-je 3, és írd ki az objektumot. Utána keresd meg ugyanígy a 99-es `id`-jű terméket is, és írd ki, mit kaptál.

12. Hányadik helyen van?

Írd ki, hányadik indexen áll a `termekek` tömbben a `Webkamera` nevű termék. Utána írd ki ugyanezt a `Nyomtató` nevű termékre is.

some és every — van-e ilyen? mind ilyen?

Mindkettő egyetlen logikai értéket (`true` / `false`) ad vissza. A `tomb.some(fn)` akkor igaz, ha **legalább egy** elemre igazat ad a függvény; a `tomb.every(fn)` akkor, ha **minden** elemre.

13. Van-e ilyen szám?

Írd ki, hogy van-e a `szamok` között 15-nél nagyobb szám. Egy második sorban írd ki, hogy van-e közöttük negatív szám.

14. Mind ilyen?

Írd ki, hogy a `szamok` tömb minden eleme pozitív-e. Egy második sorban írd ki, hogy minden eleme páros-e.

15. Készlet és árak ellenőrzése

Írd ki, hogy van-e kifogyott termék (a készlete 0). Egy második sorban írd ki, hogy minden termék ára 1000 Ft fölött van-e.

reduce — gyűjtsd egyetlen értékbe

A `tomb.reduce(fn, kezdoertek)` végigmegy a tömbön, és az elemekből **egyetlen értéket** gyűjt össze. A függvény két paramétert kap: az **eddigi eredményt** (gyűjtő) és az **aktuális elemet**; amit visszaad, az lesz a gyűjtő a következő elemnél. A gyűjtő a `kezdoertek`-ről indul, a végső értéke a `reduce` eredménye.

16. Összeg

Számold ki és írd ki a `szamok` tömb elemeinek összegét.

17. A legnagyobb

Határozd meg a `szamok` tömb legnagyobb elemét a `reduce` segítségével (a `Math.max` használata nélkül), és írd ki.

18. Készletérték

Számold ki a raktár teljes értékét: minden termék árát szorozd meg a készletével, és ezeket add össze. Írd ki az eredményt.

sort — rendezd sorba

A `tomb.sort(fn)` sorba rendezi a tömböt. Két dologra figyelj: **(1)** az **eredeti tömböt rendezi át**, ezért ha az eredetire még szükség van, előbb készíts róla másolatot; **(2)** függvény nélkül **szövegként** hasonlít, ezért számoknál mindig adj meg összehasonlító függvényt. Ez két elemet kap (`a`, `b`): ha negatív számot ad vissza, az `a` kerül előre; ha pozitívat, a `b`.

19. Növekvő és csökkenő

Készíts a `szamok` tömbből egy növekvő és egy csökkenő sorrendű új tömböt. Írd ki mindkettőt, végül az eredeti `szamok` tömböt is: annak a sorrendje nem változhat meg.

20. Nevek ábécérendben

Készíts a `nevek` tömbből egy ábécérendbe rendezett új tömböt úgy, hogy a kis- és nagybetű ne számítson (a `béla` az `Anna` és a `Csilla` közé kerüljön). Írd ki.

21. Ár szerint csökkenő

Írd ki a termékek nevét egy tömbben úgy, hogy a legdrágább termék neve legyen az első, a legolcsóbbé az utolsó. A `termekek` tömb sorrendje ne változzon.

Láncolás — több lépés egymás után

Mivel a `map`, a `filter` és a másolaton végzett `sort` is tömböt ad vissza, az eredményükön rögtön meghívható a következő metódus. Így a feldolgozás lépései egymás után, olvasható sorrendben írhatók le.

22. Raktáron lévők névsora

Írd ki egyetlen szövegként, vesszővel és szóközzel elválasztva azoknak a termékeknek a nevét, amelyekből van készleten.

MINTA KIMENET

```
<név>, <név>, <név>, …
```

Összetett feladatok

Ezeknél a feladatoknál már neked kell eldöntened, melyik metódusokra van szükség, és milyen sorrendben. Érdemes előbb szavakkal leírni a lépéseket (pl. „kiválogatom → sorba rendezem → az első hármat veszem”), és csak utána kódolni.

23. Kategóriák összesítése

Készíts egy objektumot, amely megmutatja, hogy az egyes kategóriákban hány termék van: a kulcsok a kategórianevek, az értékek a darabszámok. Írd ki az objektumot.

MINTA KIMENET

```
{ <kategória>: <darab>, <kategória>: <darab>, … }
```

24. A három legolcsóbb kapható termék

Írd ki a három legolcsóbb olyan terméket, amelyből van készleten: a legolcsóbbal kezdve, soronként egy terméket, a nevével és az árával.

MINTA KIMENET

```
<név> (<ár> Ft)
<név> (<ár> Ft)
<név> (<ár> Ft)
```

25. Áremelés a kábeleknél

Készíts egy új terméktömböt, amelyben a `kábel` kategóriájú termékek ára 10%-kal magasabb (egész forintra kerekítve), a többi termék változatlan. Az eredeti `termekek` tömb és a benne lévő objektumok nem változhatnak. Írd ki az új tömbből a kábelek nevét és új árát, végül bizonyítékként az eredeti tömbben lévő HDMI-kábel árát.

MINTA KIMENET

```
<kábel neve>: <új ár> Ft
… (minden kábelre egy sor)
Az eredeti HDMI-kábel ára: <ár> Ft
```

26. Rendelés feldolgozása

A `rendeles` tömb tételei egy termék `id`-jét és a rendelt darabszámot tartalmazzák. Írd ki minden tételről egy sorban, mi a helyzet vele (lásd a mintát): ha nincs ilyen azonosítójú termék; ha a termékből nincs elég készleten; egyébként a tétel értékét. A végén írd ki a **teljesíthető** tételek összértékét.

TOVÁBBI KIINDULÓ ADAT

```
const rendeles = [
  { id: 1, db: 2 },
  { id: 4, db: 5 },
  { id: 2, db: 1 },
  { id: 9, db: 1 }
];
```

MINTA KIMENET

```
<név>: <db> db × <ár> Ft = <érték> Ft          ← teljesíthető tétel
<név>: nincs elég készlet                      ← a készlet kisebb a rendelt darabszámnál
Ismeretlen termék (id: <id>)                   ← nincs ilyen azonosítójú termék
Fizetendő: <a teljesíthető tételek összege> Ft
```